МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»

ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И
КОМПЬЮТЕРНОЙ ТЕХНИКИ

КУРСОВАЯ РАБОТА

по дисциплине

«ИНФОРМАЦИОННЫЕ СИСТЕМЫ»

Система менеджмента персонала

Выполнили:
Рязанов Никита Сергеевич,
Софьин Вячеслав Евгеньевич,
студенты группы P3307

Проверил:
Демурчян Владимир Варламович

г. Санкт-Петербург, 2026

Содержание

Задание 3
Концептуальная модель 4
Логическая модель 6
Физическая модель 7
Реализация в PostgreSQL 8
Обеспечение целостности данных 9
    Триггеры 9
Функции и процедуры 10
Индексы 11
    Поиск вакансий по городу 11
    Поиск по набору навыков 12
    Поиск по FK 14
    Поиск и добавление предоставления решений 14
Репликация 17
Вывод 18

Задание

1. Сформировать ER-модель базы данных (на основе описаний предметной области и прецедентов из предыдущего этапа). ER-модель должна:
   - включать в себя не менее 10 сущностей;
   - содержать хотя бы одно отношение вида «многие-ко-многим».
2. Согласовать ER-модель с преподавателем. На основе ER-модели построить даталогическую модель.
3. Реализовать даталогическую модель в реляционной СУБД PostgreSQL.
4. Обеспечить целостность данных при помощи средств языка DDL и триггеров.
5. Реализовать скрипты для создания, удаления базы данных, заполнения базы тестовыми данными.
6. Предложить pl/pgsql-функции и процедуры, для выполнения критически важных запросов (которые потребуются при последующей реализации прецедентов).
7. Создать индексы на основе анализа использования базы данных в контексте описанных на первом этапе прецедентов. Обосновать полезность созданных индексов для реализации представленных на первом этапе бизнес-процессов.
8. Составить отчет.

Концептуальная модель

ER-модель предметной области содержит 20 сущностей.

Основные связи между сущностями:

- компания включает множество пользователей и команд (1:N);
- команда возглавляется одним тимлидом (1:1), включает множество сотрудников (1:N) и публикует множество вакансий (1:N);
- вакансия расположена в одном городе (N:1);
- кандидат подает множество откликов, на вакансию приходит множество откликов; отклик ссылается на одно резюме и одну анкету;
- отклик получает множество оценок HR (1:N) и одну агрегированную оценку (1:1);
- интервью связывает кандидата, тимлида и вакансию; по интервью оставляется не более одного отзыва (1:1);
- задача имеет множество тестов (1:N) и одного автора (N:1);
- посылка относится к одному отправителю, контесту, задаче и языку программирования (N:1);
- пользователь имеет множество отчетов о GitHub-активности (1:N).

Отношения вида «многие-ко-многим»:

- `Vacancy` – `Skill`: вакансия требует набор навыков, навык востребован во многих вакансиях;
- `Form` – `Skill`: кандидат указывает в анкете навыки, которыми владеет;
- `Contest` – `Task`: контест состоит из нескольких задач, задача может использоваться в разных контестах;
- `Vacancy` – `Contest`: к вакансии привязываются отборочные контесты, один контест может использоваться для нескольких вакансий;
- `Contest` – `Person`: участники контеста.

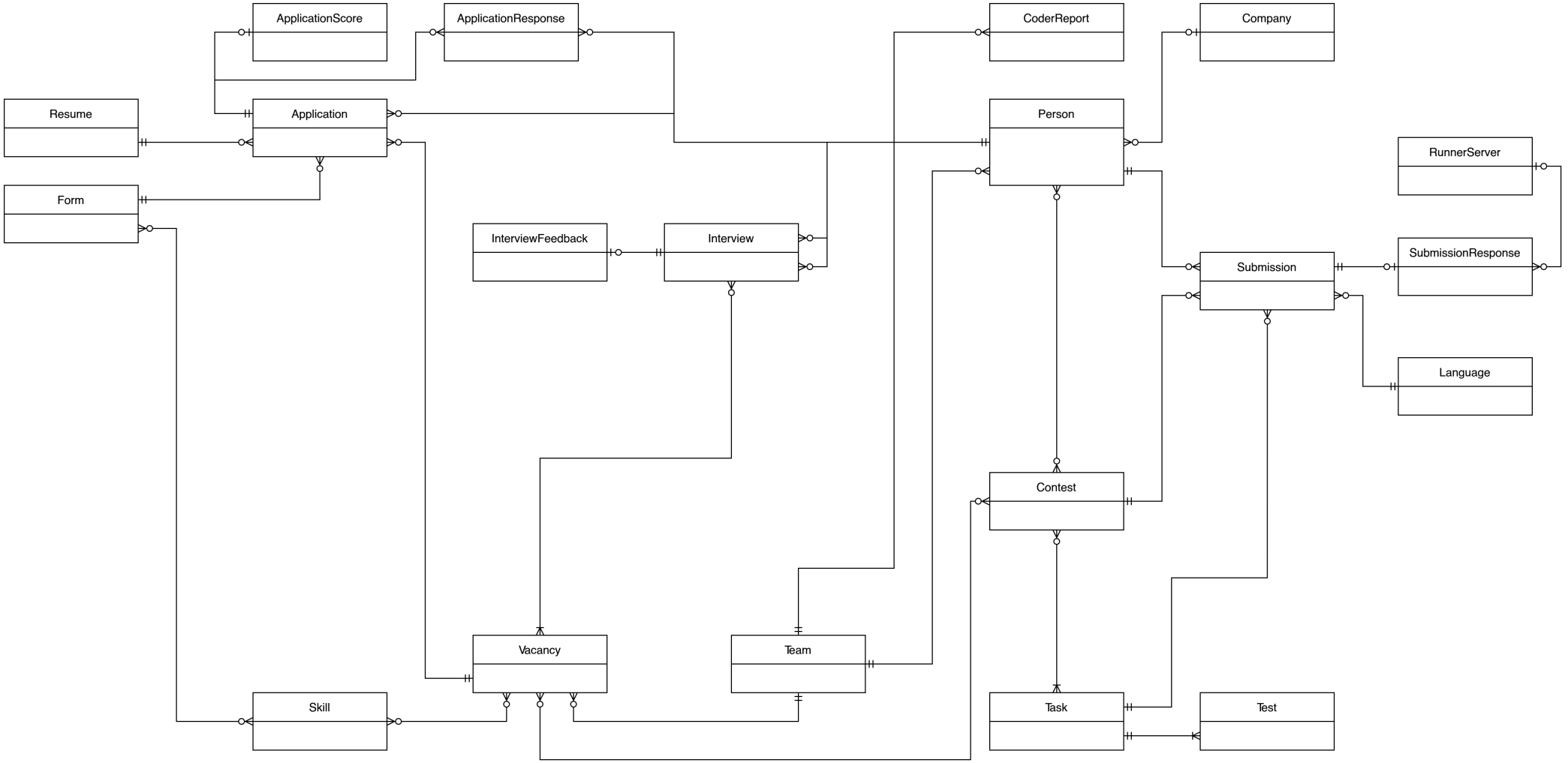


Рис. 1. Концептуальная модель

Логическая модель

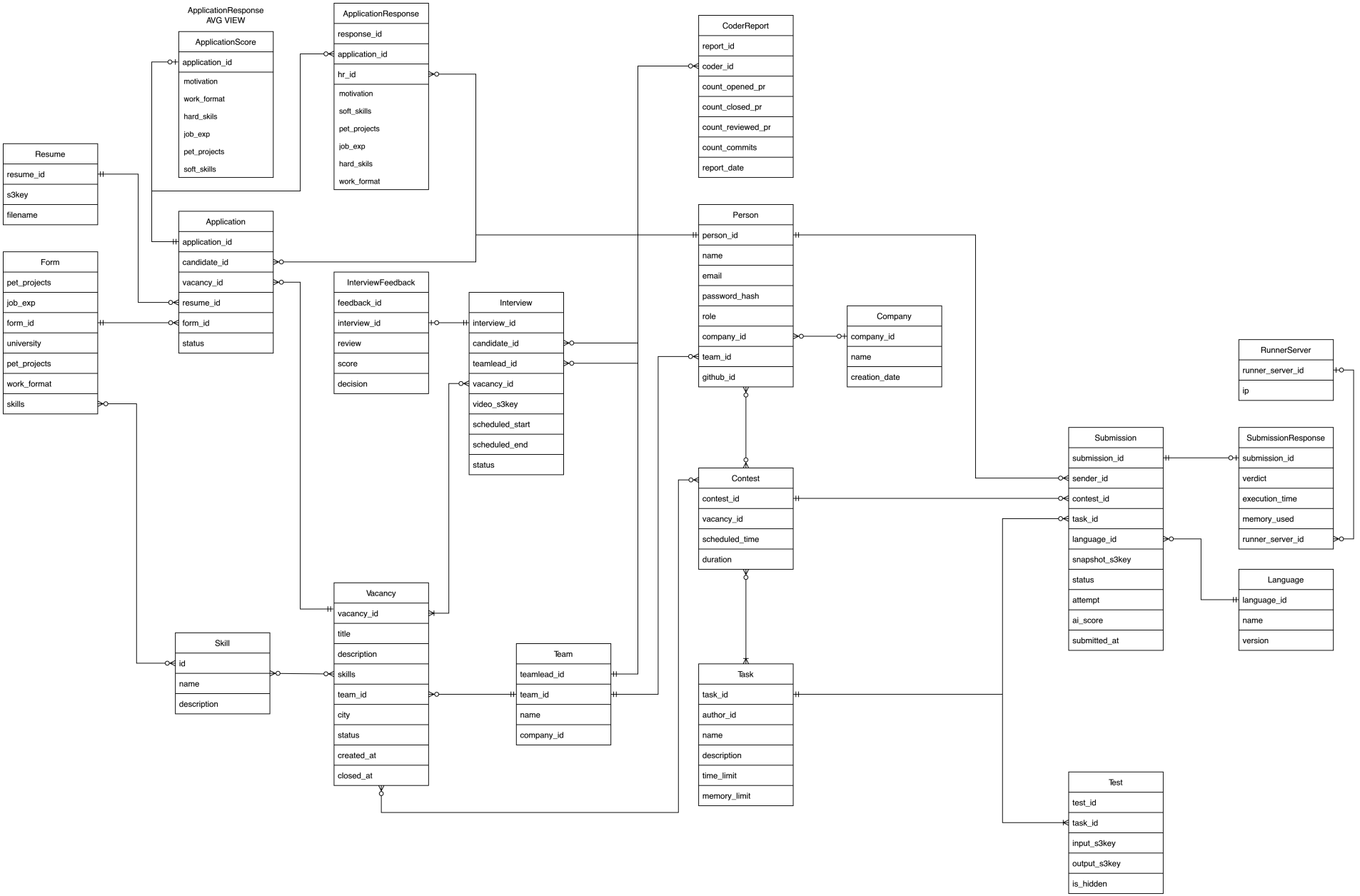


Рис. 2. Логическая модель

Физическая модель

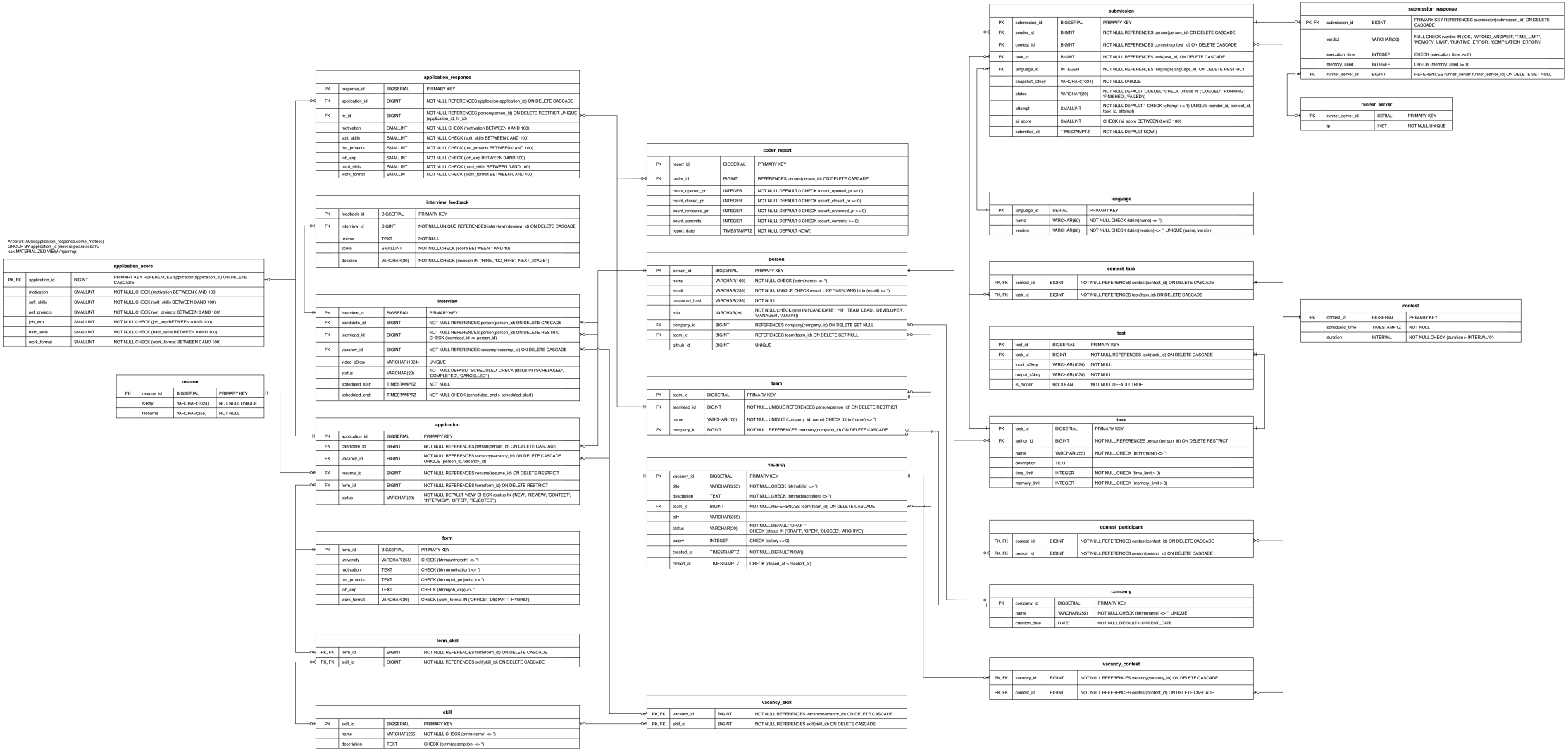


Рис. 3. Логическая модель

Реализация в PostgreSQL

Физическая модель реализована в СУБД PostgreSQL.

Скрипты расположены в репозитории проекта:
https://github.com/safarislava/itmo-team

Скрипт	**Назначение**
`create.sql`	Создание таблиц и ограничений целостности
`func.sql`	Функции, процедуры, триггеры и задание планировщика `pg_cron`
`index.sql`	Создание индексов
`fill.sql`	Заполнение базы тестовыми данными
`truncate.sql`	Очистка всех таблиц со сбросом последовательностей
`drop.sql`	Удаление всех таблиц

Таблица 1. Скрипты базы данных

База данных разворачивается последовательным выполнением скриптов.

Для работы скрипта `func.sql` требуется расширение `pg_cron`, которое должно быть подключено на сервере PostgreSQL.

Обеспечение целостности данных

Триггеры

Ограничения, которые невозможно выразить декларативно (зависящие от данных других таблиц), реализованы триггерами:

Триггер таблица, событие	**Назначение**
`trg_check_team_teamlead_id` `team`, `BEFORE INSERT OR UPDATE`	Проверяет, что руководитель команды имеет роль `TEAM_LEAD`.
`trg_check_application_response_hr_id` `application_response`, `BEFORE INSERT OR UPDATE`	Проверяет, что оценку отклику выставляет пользователь с ролью `HR`.
`trg_check_application_candidate_id` `application`, `BEFORE INSERT OR UPDATE`	Проверяет, что отклик подает пользователь с ролью `CANDIDATE`.
`trg_check_interview_candidate_id` `interview`, `BEFORE INSERT OR UPDATE`	Проверяет, что интервью назначается пользователю с ролью `CANDIDATE`.
`trg_check_application_for_open_vacancy` `application`, `BEFORE INSERT`	Запрещает отклик на вакансию, не находящуюся в статусе `OPEN`.
`trg_increment_attempt_counter` `submission`, `BEFORE INSERT`	Автоматически вычисляет номер попытки как следующий за максимальным для тройки «отправитель – контест – задача».

`trg_refresh_application_score`
`application_response`,
`AFTER INSERT OR UPDATE OR DELETE`
	Пересчитывает агрегированные оценки откликов в `application_score` как средние значения оценок HR.

Таблица 2. Триггеры

Функции и процедуры

Для критически важных запросов, необходимых при реализации прецедентов, на языке PL/pgSQL реализованы функции и процедуры:

Функция / процедура	**Назначение**
`archive_vacancies` процедура	Перевод вакансий, закрытых более 6 месяцев назад, в статус `ARCHIVE`. Запускается ежедневно в 00:00 через `pg_cron`.
`close_vacancy_with_rejections` функция	Закрытие вакансии и отклонение всех откликов на нее, кроме получивших оффер. Возвращает идентификаторы отклоненных откликов для уведомления кандидатов.
`get_candidate_leaderboard` функция	Рейтинг кандидатов на вакансию: рекомендательный балл — взвешенная сумма агрегированных оценок HR с весами, заданными тимлидом.
`schedule_interview` процедура	Назначение интервью по отклику: проверка, что у тимлида нет пересекающихся интервью, создание интервью и перевод отклика в статус `INTERVIEW`.

Таблица 3. Функции и процедуры

Индексы

Поиск вакансий по городу

Наиболее вероятно, что данные заполняются 1 раз и больше не обновляются. Преобладает нагрузка чтения.

Выделим потенциально часто используемые запросы высокой и низкой селективности.

```
SELECT * FROM vacancy WHERE city = 'Санкт-Петербург';
SELECT * FROM vacancy WHERE city = 'Воронеж';
```

Селективность	Индекс	Scan	Cost	Planning	Execution
Низкая	Без индекса	Seq Scan	0.00 .. 2360.00	0.220	12.550
	B-Tree	Bitmap Heap Scan	44.80 .. 2261.60	0.769	9.903
	Hash	Seq Scan	0.00 .. 2360.00	0.180	10.115
Высокая	Без индекса	Seq Scan	0.00 .. 2351.25	0.947	9.620
	B-Tree	Bitmap Heap Scan	4.40 .. 60.80	0.123	0.328
	Hash	Bitmap Heap Scan	4.12 .. 60.51	0.389	0.574

Видно, что при B-Tree всегда применяется не последовательный режим сканирования и запросы выполняются быстрее всего.

Также B-Tree способен помочь при неточном поиске, что, очевидно, невозможно при хеш-индексе.

```
SELECT * FROM vacancy WHERE city ~ '^Воро';
```

Индекс	Scan	Cost	Planning	Execution
Без индекса	Seq Scan	0.00 .. 2602.00	0.665	24.182
С индексом	Seq Scan	0.00 .. 2602.00	0.104	12.391

Поиск по набору навыков

Навыки используются в 2 UseCase'ах: при поиске вакансий кандидатом и для заполнения отклика и дальнейшей фильтрации по ним тимлидом. В обоих вариантах наиболее вероятно, что данные заполняются 1 раз и больше не обновляются. Преобладает нагрузка чтения.

Рассмотрим на примере поиска вакансий.

Изначально предполагался такой запрос

```
SELECT * FROM match_vacancies(ARRAY[2, 3, 4]);
```

Индекс	Scan	Cost	Planning	Execution
Без индекса	Function Scan	0.25 .. 10.25	0.066	49.489
(skill_id, vacancy_id)	Function Scan	0.25 .. 10.25	0.066	57.789

Стало только хуже. Оказалось Postgres не умеет использовать индексы для pgplsql функций.

Можно составлять запрос динамически на бэкенде:

```
SELECT v.vacancy_id, v.title, v.salary
FROM vacancy v
WHERE v.status = 'OPEN'
  AND v.vacancy_id IN (
    SELECT vacancy_id FROM vacancy_skill WHERE skill_id = 2
    INTERSECT
    SELECT vacancy_id FROM vacancy_skill WHERE skill_id = 3
    INTERSECT
    SELECT vacancy_id FROM vacancy_skill WHERE skill_id = 4
);
Hash Join  (cost=2402.68..6311.33 rows=266 width=35) (actual time=31.581..31.607
rows=5 loops=1)
  ...
"  Hash Cond: (""ANY_subquery"".vacancy_id = v.vacancy_id)"
""*SELECT* 1""  (cost=0.00..1234.46 rows=1282 width=12) (actual time=5.465..8.111
rows=1284 loops=1)"
  ...

""*SELECT* 2""  (cost=0.00..1278.69 rows=5705 width=12) (actual time=0.287..6.175
rows=5666 loops=1)"
  ...
"                    ->  Subquery Scan on ""*SELECT* 3""  (cost=0.00..1276.69
rows=5505 width=12) (actual time=0.066..4.690 rows=5534 loops=1)"
```

```
  ...
  ->  Hash  (cost=2351.25..2351.25 rows=4114 width=35) (actual time=9.543..9.543
rows=4114 loops=1)
        Buckets: 8192  Batches: 1  Memory Usage: 342kB
        ->  Seq Scan on vacancy v  (cost=0.00..2351.25 rows=4114 width=35) (actual
time=0.035..8.666 rows=4114 loops=1)
              Filter: ((status)::text = 'OPEN'::text)
              Rows Removed by Filter: 15186
Planning Time: 1.148 ms
Execution Time: 31.961 ms
```

Здесь подзапросы выполняются параллельно, что ускоряет доступ.

Можно ещё добавить такие индексы, которые при тестировании добавлялись последовательно, при этом каждый улучшал время выполнения:

```
CREATE INDEX idx_vacancy_skill_skill_id ON vacancy_skill (skill_id, vacancy_id);
CREATE INDEX idx_vacancy_status_open ON vacancy (vacancy_id) WHERE status = 'OPEN';
Hash Join  (cost=1922.18..4107.04 rows=266 width=35) (actual time=12.069..12.610
rows=5 loops=1)
"  Hash Cond: (v.vacancy_id = ""ANY_subquery"".vacancy_id)"
  ->  Bitmap Heap Scan on vacancy v  (cost=77.88..2251.94 rows=4114 width=35) (actual
time=1.989..5.562 rows=4114 loops=1)
Planning Time: 0.402 ms
Execution Time: 13.453 ms
```

Время выполнения ещё улучшилось.

Кроме того, можно денормализовать таблицу `vacancy` и добавить в неё массив требуемых навыков. Тогда не нужна таблица `vacancy_skill` и `JOIN` при запросах.

```
SELECT vacancy_id, title, salary FROM vacancy WHERE status = 'OPEN' AND skill_ids @>
ARRAY[2, 3, 4]::bigint[];
```

Добавим GIN-индекс для скорости.

```
CREATE INDEX idx_vacancy_skills_open ON vacancy USING gin (skill_ids) WHERE status =
'OPEN';
```

И работает это blazing fast!!!

```
Bitmap Heap Scan on vacancy  (cost=17.35..32.91 rows=4 width=35) (actual
time=0.506..0.507 rows=0 loops=1)
"  Recheck Cond: ((skill_ids @> '{2,3,4}'::bigint[]) AND ((status)::text =
'OPEN'::text))"
  ->  Bitmap Index Scan on idx_vacancy_skills_open  (cost=0.00..17.35 rows=4 width=0)
```

```
(actual time=0.427..0.428 rows=0 loops=1)
"        Index Cond: (skill_ids @> '{2,3,4}'::bigint[])"
Planning Time: 0.220 ms
Execution Time: 0.823 ms
```

Поиск по FK

Выделены UseCase'ы, в которых надо искать по внешнему ключу: вакансии по команде, отклики по вакансии, интервью по кандидату, тесты по заданию. В некоторых сценариях нужны дополнительные проверки, но тут выделены с наивысшей селективностью. Во всех этих сценариях преобладает нагрузка по чтению.

Проведём тесты на первом из них – вакансии по команде.

```
SELECT * FROM vacancy WHERE team_id = 10;
```

Индекс	Scan	Cost	Planning	Execution
Без индекса	Seq Scan	0.00 .. 2602.00	0.403	23.461
Hash	Bitmap Heap Scan	4.08 .. 42.25	0.347	0.328
B-Tree	Bitmap Heap Scan	4.37 .. 42.53	0.257	0.311

Время выполнения примерно одинаковое. Так что можно выбрать любой из индексов. Тогда выберем для них B-Tree, так как это оставит возможность составных ключей и оптимизации при сортировке.

Поиск и добавление предоставления решений

При добавлении ограничении `UNIQUE` добавляется B-Tree индекс.

```
CONSTRAINT uq_submission_attempt UNIQUE (sender_id, contest_id, task_id, attempt)
SELECT * FROM submission WHERE sender_id = 102326;
```

Индекс	Scan	Cost	Planning	Execution
Без индекса	Parallel Seq Scan	1000.00 .. 30978.77	0.088	46.964
С индексом	Bitmap Heap Scan	4.76 .. 171.69	1.603	0.901

На чтение ограничение влияет положительно.

Оценим влияние на запись.

```
BEGIN;
EXPLAIN ANALYSE
```

```
INSERT INTO submission (sender_id, contest_id, task_id, language_id, snapshot_s3key)
SELECT
    cp.person_id,
    cp.contest_id,
    ct.task_id,
    (SELECT language_id FROM language LIMIT 1),
    'submissions/bench/' || gen_random_uuid() || '.py'
FROM contest_participant cp
         JOIN contest_task ct ON ct.contest_id = cp.contest_id
LIMIT 100;
ROLLBACK
```

Без ограничения

```
Insert on submission  (cost=0.41..5.67 rows=0 width=0) (actual time=6857.461..6857.491
rows=0 loops=1)
"  ->  Subquery Scan on ""*SELECT*""  (cost=0.41..5.67 rows=100 width=622) (actual
time=0.411..5.280 rows=100 loops=1)"
              InitPlan 1 (returns $0)
                ->  Limit  (cost=0.00..0.11 rows=1 width=4) (actual time=0.008..0.036
rows=1 loops=1)
                      ->  Seq Scan on language  (cost=0.00..1.10 rows=10 width=4)
(actual time=0.007..0.011 rows=1 loops=1)
              ->  Nested Loop  (cost=0.30..15928.40 rows=353203 width=60) (actual
time=0.396..4.489 rows=100 loops=1)
                    ->  Seq Scan on contest_participant cp  (cost=0.00..1072.64
rows=69564 width=16) (actual time=0.022..0.044 rows=21 loops=1)
Planning Time: 1.333 ms
Trigger for constraint submission_sender_id_fkey: time=7.703 calls=100
Trigger for constraint submission_contest_id_fkey: time=1.570 calls=100
Trigger for constraint submission_task_id_fkey: time=1.840 calls=100
Trigger for constraint submission_language_id_fkey: time=0.473 calls=100
Trigger trg_increment_attempt_counter: time=6819.141 calls=100
Execution Time: 6870.402 ms
```

С ограничением:

```
Trigger for constraint submission_task_id_fkey: time=2.102 calls=100
Trigger for constraint submission_language_id_fkey: time=0.661 calls=100
Trigger trg_increment_attempt_counter: time=4.042 calls=100
Execution Time: 15.419 ms
```

Объясняется это тем, что есть триггер автоинкрементирующий счётчик попытки. Внутри он с помощью `Seq Scan` ищет текущий максимум.

В этом случае ограничение улучшает согласованность, удобство использования и производительность.

Репликация

В PostgreSQL репликация относится к типу master – slave. Мы реализуем схему 1 – 2 для повышения производительности и отказоустойчивости системы.

Репликация будет происходить в асинхронном формате, когда master не дожидается применения записей slave'ом. По CAP теореме это увеличит доступность системы, но жертвует согласованностью.

В PostgresSQL по умолчанию нет механизма смены master после его сбоя. Для решения этой проблемы будем использовать Patroni. Это увеличит отказоустойчивость системы, а также доступность, но может привести к потере данных.

Также Patroni умеет реплицировать и данные, и схему данных, что недоступно по умолчанию при логической репликации Postgres и не требует одинаковых версии ПО и ОС и архитектуры оборудования.

Дополнительно для Patroni надо поднять etcd, чтобы обеспечить высокую согласованность принятых кворумом решений.

Для борьбы с устаревшими данными разделим UseCase'ы по CQRS:

1. UseCase'ы чтения – выполняются на slave.
2. UseCase'ы с записью – выполняются на master.

Запросы с записью должны возвращать обновлённое состояние, чтобы избежать запроса чтения, который с большой вероятностью вернёт устаревшие данные с slave'а.

Итого: на всех трёх нодах развёзрнут PostgresSQL, etcd, Patroni. При такой конфигурации нет точки единого отказа и выполняется условие строго кворума. Подробнее конфигурационные файлы можно изучить в репозитории:

https://github.com/safarislava/itmo-team/tree/main/deployment

Вывод

В ходе второго этапа курсовой работы на основе описания предметной области и прецедентов первого этапа построены концептуальная, логическая и физическая модели.

Модель реализована в СУБД PostgreSQL. Подготовлены скрипты создания, удаления, очистки и заполнения базы тестовыми данными. Для критически важных операций реализованы функции и процедуры на PL/pgSQL, которые будут использованы на уровне хранения информационной системы на следующем этапе. Была настроена логическая репликация базы данных для обеспечения отказоустойчивости, резервного копирования и распределения нагрузки на чтение. Кроме того, были созданы индексы, ускоряющие частые запросы основных прецедентов.